

□ Bài giảng về phân tích mạch khuếch đại dùng JFET

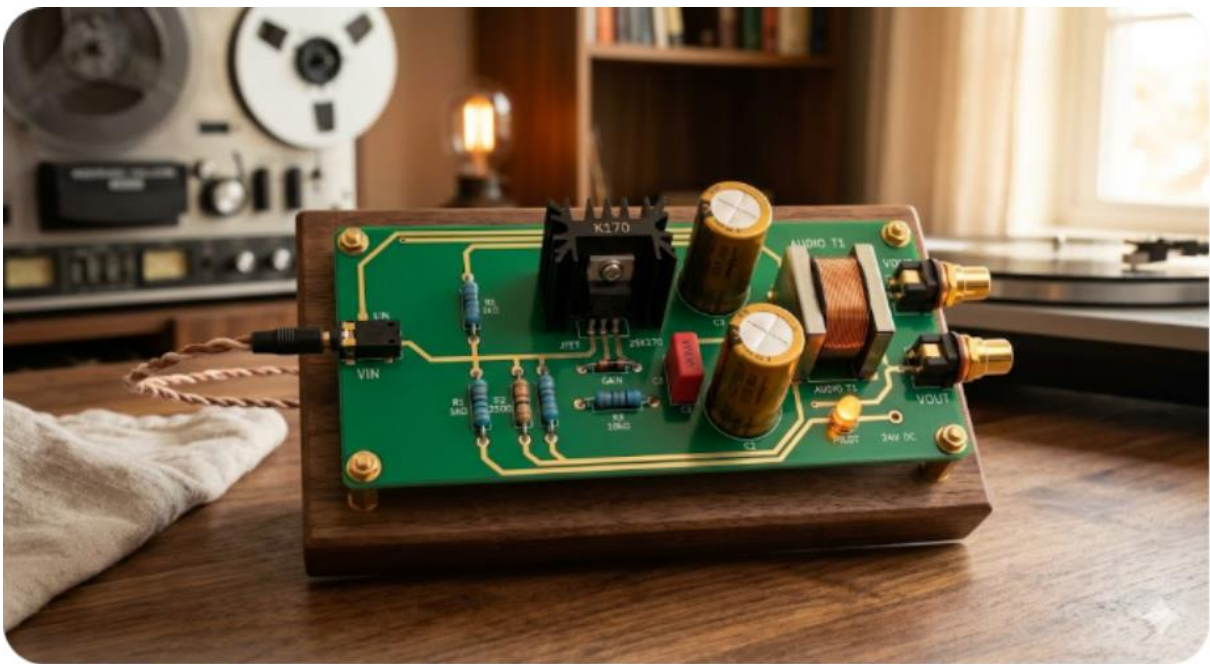
Sưu tầm và Biên soạn bổ sung phần Vibe Coding

With the support of Gemini and A/Prof Tran Thu Ha (HCMUTE)

Prof Happy AI

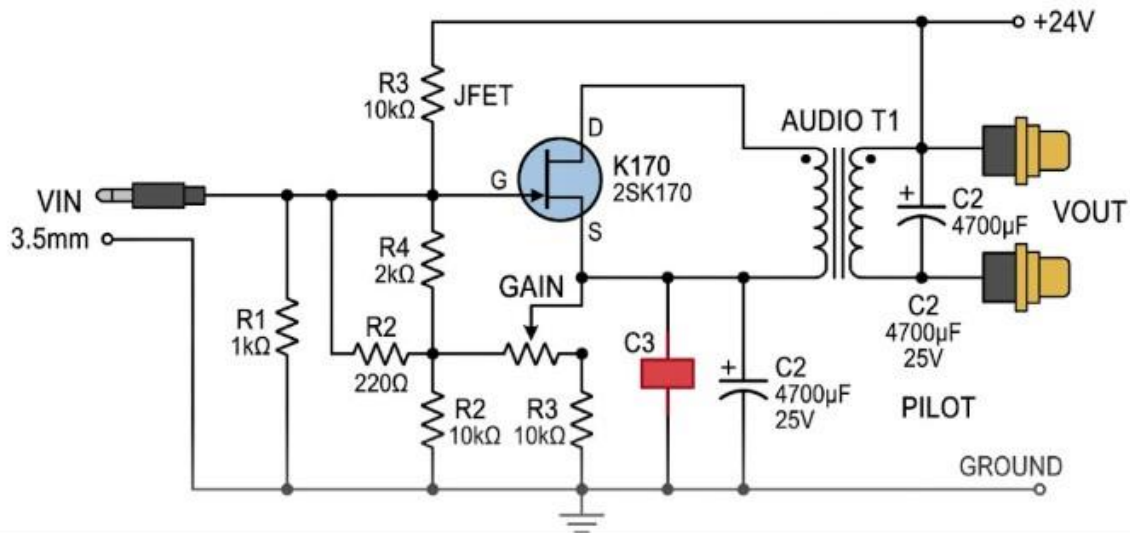
Email: phucdo18@gmail.com

Date: May 10, 2026



□ Mạch điện và giải thích:

Sơ đồ mạch điện khuếch đại JFET 2SK170



Mạch điện bạn cung cấp là một mạch khuếch đại âm thanh sử dụng **JFET N-channel 2SK170**, một linh kiện nổi tiếng với độ nhiễu cực thấp, thường dùng trong các tầng tiền khuếch đại (pre-amp) chất lượng cao.

Dưới đây là phân tích chi tiết về sơ đồ này:

1. Thành phần cấu tạo chính

- **JFET 2SK170 (K170):** Trái tim của mạch, đóng vai trò là phần tử khuếch đại chính.
- **Biến áp AUDIO T1:** Đây là điểm đặc biệt. Thay vì dùng tụ điện để lấy tín hiệu ra, mạch dùng biến áp cách ly để phối hợp trở kháng và truyền tín hiệu sang tải (VOUT).
- **Hệ thống phân cực (R_3 , R_4 , R_2):** Các điện trở này thiết lập điện áp tại cực Cổng (Gate - G) và cực Nguồn (Source - S) để đưa JFET vào vùng hoạt động tuyến tính.
- **Tụ lọc nguồn (C_2):** Các tụ hóa lớn ($4700\mu F$) giúp làm phẳng điện áp DC từ nguồn $+24V$, đảm bảo mạch hoạt động ổn định và không bị nhiễu xoay chiều (hum).

2. Nguyên lý hoạt động

Mạch này hoạt động theo cấu hình **Cực nguồn chung (Common Source)** nhưng có biến tấu ở phần tải ngõ ra.

- **Ngõ vào (VIN):** Tín hiệu âm thanh từ jack $3.5mm$ đi qua điện trở R_1 ($1k\Omega$) để vào cực G của JFET. R_1 đóng vai trò ổn định trở kháng vào.

- **Điều chỉnh độ lợi (GAIN):** Biến trở tại cực S cho phép bạn thay đổi giá trị điện trở thoát cực nguồn.
- Khi điện trở này nhỏ, độ lợi khuếch đại sẽ tăng cao.
- Tụ C_3 nối song song giúp thoát tín hiệu AC, làm tăng đáng kể độ lợi áp của mạch.
- **Ngõ ra (VOUT):** Dòng điện cực Máng (Drain - D) thay đổi theo tín hiệu vào, chạy qua cuộn sơ cấp của biến áp $T1$. Biến áp sẽ cảm ứng sang cuộn thứ cấp để đưa tín hiệu ra loa hoặc tai nghe tại cổng VOUT.

3. Phân tích ưu điểm của mạch này

- **Độ nhiễu cực thấp:** Nhờ sử dụng 2SK170, mạch này rất thích hợp cho các tín hiệu nhỏ như micro hoặc đầu phát nhạc chất lượng cao.
- **Cách ly tín hiệu:** Việc sử dụng biến áp $T1$ giúp cách ly hoàn toàn điện áp một chiều (DC) giữa mạch khuếch đại và thiết bị đầu ra, bảo vệ loa/tai nghe khỏi các sự cố chạm chập DC.
- **Chất âm đặc trưng:** Sự kết hợp giữa JFET (có đặc tính âm thanh gần giống đèn điện tử chân không) và biến áp xuất âm thường tạo ra chất âm ấm áp, mượt mà, rất được giới chơi âm thanh (audiophile) ưa chuộng.

4. Thông số cần lưu ý

- **Nguồn cấp:** $+24V$ là mức điện áp khá cao cho một mạch JFET đơn, cho phép dải động (headroom) của tín hiệu lớn, ít bị méo tiếng khi mở âm lượng cao.
- **Tụ điện:** Việc dùng các tụ $4700\mu F$ cho thấy thiết kế này rất chú trọng vào độ sạch của nguồn điện.

Mạch điện này không chỉ là một bài học lý thuyết mà là một thiết kế thực dụng cao, thiên về hướng thưởng thức âm thanh chất lượng. Bạn có đang dự định lắp ráp mạch này trên bo test không?

Có thể ứng dụng lập trình Python để giải quyết các bài toán phân tích từ cơ bản đến nâng cao cho mạch JFET này. Dưới đây là một số bài toán lập trình cụ thể:

1. Phân tích điểm làm việc tĩnh (DC Operating Point) Bài toán này sử dụng các phương pháp số học để tìm giao điểm của đường tải DC và đặc tuyến truyền đạt của JFET 2SK170. Nhiệm vụ: Giải phương trình phi tuyến Shockley kết hợp với

phương trình đường thẳng phân cực của mạch. Thư viện: `scipy.optimize` (hàm `fsolve`). Mục tiêu: Xác định giá trị chính xác của I_{DQ} và V_{GSQ} khi thay đổi các điện trở phân cực R_3, R_4 .

2. Mô phỏng đáp ứng tần số (AC Analysis) Đây là bài toán cốt lõi để đánh giá chất lượng âm thanh của mạch, đặc biệt là ảnh hưởng của biến áp $T1$ và các tụ điện. Nhiệm vụ: Tính toán độ lợi điện áp A_v theo tần số f (từ $20Hz$ đến $20kHz$). Thư viện: `numpy`, `matplotlib`. Nội dung: Lập trình hàm truyền (Transfer Function) dựa trên mô hình tương đương tín hiệu nhỏ, bao gồm cả cảm kháng của biến áp và dung kháng của các tụ C_2, C_3 .
3. Phân tích độ nhạy (Sensitivity Analysis) Trong nghiên cứu, việc hiểu rõ sự thay đổi của linh kiện ảnh hưởng thế nào đến hệ thống là rất quan trọng. Nhiệm vụ: Đánh giá sự thay đổi của độ lợi (Gain) khi các thông số như I_{DSS} hoặc V_P của JFET thay đổi do sai số sản xuất. Phương pháp: Sử dụng mô phỏng Monte Carlo để chạy hàng nghìn kịch bản với các giá trị linh kiện ngẫu nhiên trong dải sai số cho phép (ví dụ 5%).
4. Chuyển đổi sơ đồ mạch sang cấu trúc đồ thị (Graph-based Analysis) Dựa trên mối quan tâm của bạn về Graph Machine Learning, bạn có thể thực hiện một bài toán nâng cao: Nhiệm vụ: Biểu diễn sơ đồ mạch điện dưới dạng một đồ thị (Graph) với các nút (nodes) là các điểm nối và cạnh (edges) là các linh kiện. Thư viện: `NetworkX`. Ứng dụng: Tự động hóa việc lập phương trình nút (Nodal Analysis) để giải mạch điện bất kỳ bằng Python.
5. Tối ưu hóa thông số (Optimization Problem) Nhiệm vụ: Tìm giá trị tối ưu của các điện trở và biến trở GAIN để đạt được độ lợi mong muốn mà vẫn giữ được độ méo tiếng thấp nhất. Thư viện: `scipy.optimize` (thuật toán `minimize`).

□ Chương trình 1:

Phân tích điểm làm việc tĩnh (DC Operating Point):

Bài toán này sử dụng các phương pháp số học để tìm giao điểm của đường tải DC và đặc tuyến truyền đạt của JFET 2SK170. Nhiệm vụ: Giải phương trình phi tuyến Shockley kết hợp với phương trình đường thẳng phân cực của mạch.

Thư viện: `scipy.optimize` (hàm `fsolve`).

Mục tiêu: Xác định giá trị chính xác của I_{DQ} và V_{GSQ} khi thay đổi các điện trở phân cực R_3, R_4 .

Để giải bài toán này, chúng ta sẽ kết hợp phương trình đặc tuyến JFET (Shockley) và phương trình đường tải DC thiết lập bởi mạch phân cực cổng và nguồn. Dưới đây là

đoạn mã Python sử dụng `scipy.optimize.fsolve` để tìm điểm làm việc tĩnh $Q(V_{GSQ}, I_{DQ})$.

```
import numpy as np
from scipy.optimize import fsolve

def solve_dc_bias(R3, R4, R2_top, R2_bot, RS_gain, VDD, IDSS, VP):
    """
    Tìm điểm làm việc tĩnh Q cho mạch JFET.

    Thông số:
    R3, R4: Điện trở phân áp tại cổng.
    R2_top, R2_bot: Các điện trở phần divider của mạch phân áp/phản hồi.
    RS_gain: Giá trị tổng trở tại cực Nguồn (Source) - tính cả biến trở GAIN
    VDD: Nguồn cấp (V).
    IDSS: Dòng điện bão hòa cực máng (A).
    VP: Điện áp ngắt (V).
    """

    # 1. Tính điện áp divider divider tại cực Cổng (VG) bằng định luật phân áp
    # Giả sử dòng qua Gate bằng 0
    VG = VDD * (R4 / (R3 + R4))

    def equations(p):
        VGS, ID = p

        # Phương trình 1: Phương trình Shockley (Đặc tuyến JFET)
        # ID = IDSS * (1 - VGS/VP)^2
        f1 = IDSS * (1 - VGS/VP)**2 - ID

        # Phương trình 2: Phương trình phân cực (Kỹ thuật Kirchhoff cho vòng Gate-S)
        # VG - VGS - ID*RS = 0 => VGS = VG - ID*RS
        f2 = VG - ID * RS_gain - VGS

        return [f1, f2]

    # Dự đoán ban đầu (VGS gần bằng 0, ID bằng một nửa IDSS)
    initial_guess = [VP/2, IDSS/2]

    VGSQ, IDQ = fsolve(equations, initial_guess)
    return VGSQ, IDQ

# --- THÔNG SỐ GIẢ ĐỊNH CHO 2SK170 ---
VDD_val = 24.0      # Nguồn 24V
IDSS_val = 0.010    # 10mA (Dòng IDSS điển hình của dòng 2SK170BL)
VP_val = -1.5       # Điện áp ngắt (V)

# Thông số điện trở từ sơ đồ (Ước lượng từ hình ảnh)
R3_val = 10000      # 10k
R4_val = 2000       # 2k
RS_val = 500        # Giả sử biến trở GAIN đang ở mức 500 Ohm

# Giải bài toán
```

```

vgsq, idq = solve_dc_bias(R3_val, R4_val, 0, 0, RS_val, VDD_val, IDSS_val, V

print(f"--- KẾT QUẢ PHÂN TÍCH DC ---")
print(f"Điện áp cực Cổng-Nguồn (VGSQ): {vgsq:.4f} V")
print(f"Dòng điện cực Máng (IDQ):      {idq * 1000:.4f} mA")
print(f"Trạng thái: {'Hoạt động' if VP_val < vgsq < 0 else 'Khóa/Bão hòa'}")

```

```

--- KẾT QUẢ PHÂN TÍCH DC ---
Điện áp cực Cổng-Nguồn (VGSQ): -0.1358 V
Dòng điện cực Máng (IDQ):      8.2716 mA
Trạng thái: Hoạt động

```

Phân tích thuật toán:

Hệ phương trình phi tuyến: * Mạch JFET không thể giải bằng các phép tính tuyến tính đơn giản vì dòng I_D tỉ lệ với bình phương của V_{GS} .

Hàm equations định nghĩa sai số giữa mô hình lý thuyết và thực tế mạch điện. fsolve sẽ tìm cách triệt tiêu sai số này về 0.

Thông số linh kiện: * I_{DSS} và V_P của 2SK170 thường thay đổi theo phân loại (Grade R, O, BL, V). Trong code, tôi sử dụng thông số dòng BL (thông dụng nhất).

Khả năng mở rộng: Bạn có thể đặt đoạn code này vào một vòng lặp for để quét (sweep) giá trị của điện trở R_4 hoặc biến trở GAIN.

Việc này giúp bạn vẽ được đồ thị đường đặc tuyến truyền đạt và tìm ra vùng làm việc tuyến tính nhất cho tín hiệu âm thanh, tránh hiện tượng méo dạng (clipping).

Phương pháp giải số học này chính xác hơn nhiều so với việc ước lượng bằng đồ thị thủ công, đặc biệt khi bạn cần thiết kế các tầng khuếch đại có độ nhạy cao.

□ Biểu diễn trực quan

Để biểu diễn trực quan kết quả của chương trình phân tích điểm làm việc tĩnh (Q-point), chúng ta cần vẽ đồ thị mô tả sự giao thoa giữa Đặc tuyến truyền đạt (phương trình Shockley) và Đường thẳng phân cực (phương trình Kirchhoff).

Dưới đây là đoạn mã bổ sung sử dụng matplotlib để trực quan hóa kết quả:

```

import matplotlib.pyplot as plt

# 1. Tạo dữ liệu cho đặc tuyến Shockley
vgs_range = np.linspace(VP_val, 0, 100)
id_shockley = IDSS_val * (1 - vgs_range / VP_val)**2

# 2. Tạo dữ liệu cho đường thẳng phân cực (Bias Line)
# Từ phương trình: VGS = VG - ID * RS => ID = (VG - VGS) / RS

```

```

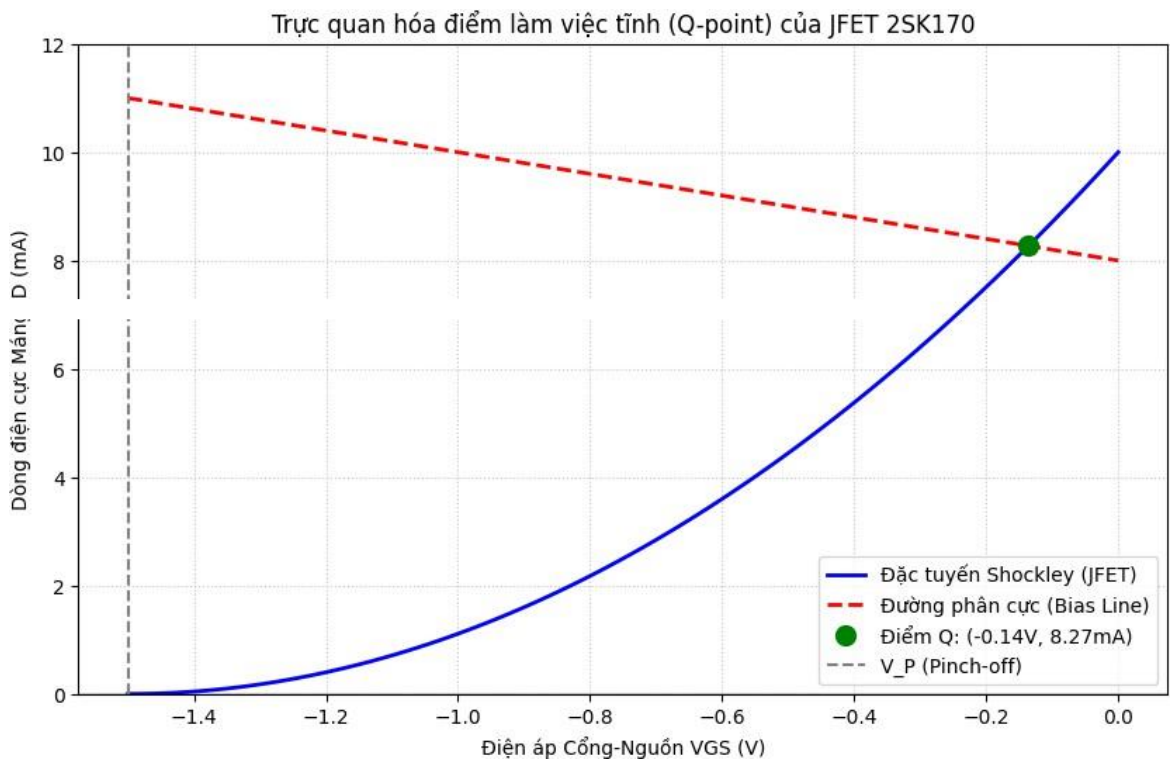
VG_val = VDD_val * (R4_val / (R3_val + R4_val))
id_bias_line = (VG_val - vgs_range) / RS_val

# 3. Vẽ đồ thị
plt.figure(figsize=(10, 6))
plt.plot(vgs_range, id_shockley * 1000, 'b-', lw=2, label='Đặc tuyến Shockle')
plt.plot(vgs_range, id_bias_line * 1000, 'r--', lw=2, label='Đffờng phân cực')

# Đánh dấu điểm Q
plt.plot(vgsq, idq * 1000, 'go', markersize=10, label=f'Điểm Q: ({vgsq:.2f}V

# Định dạng đồ thị
plt.title('Trực quan hóa điểm làm việc tĩnh (Q-point) của JFET 2SK170')
plt.xlabel('Điện áp Cổng-Nguồn VGS (V)')
plt.ylabel('Dòng điện cực Máng ID (mA)')
plt.grid(True, linestyle=':', alpha=0.7)
plt.axhline(0, color='black', lw=1)
plt.axvline(VP_val, color='gray', linestyle='--', label='V_P (Pinch-off)')
plt.legend()
plt.ylim(0, IDSS_val * 1200) # Giới hạn trục Y để dễ quan sát
plt.show()

```



□ Chương trình 2

Mô phỏng đáp ứng tần số (AC Analysis) Đây là bài toán cốt lõi để đánh giá chất lượng âm thanh của mạch, đặc biệt là ảnh hưởng của biến áp T1 và các tụ điện. Nhiệm vụ:

Tính toán độ lợi điện áp A_v theo tần số f (từ 20Hz đến 20kHz).

Thư viện: numpy, matplotlib.

Nội dung: Lập trình hàm truyền (Transfer Function) dựa trên mô hình tương đương tín hiệu nhỏ, bao gồm cả cảm kháng của biến áp và dung kháng của các tụ C2, C3 .

Để mô phỏng đáp ứng tần số (AC Analysis) cho mạch khuếch đại này, chúng ta sẽ sử dụng mô hình tín hiệu nhỏ của JFET kết hợp với các linh kiện thụ động. Bài toán này tập trung vào việc tính toán sự thay đổi của độ lợi (Gain) theo tần số, bị ảnh hưởng bởi dung kháng của tụ điện ($1/j\omega C$) và cảm kháng của biến áp ($j\omega L$).

Dưới đây là mã nguồn Python để mô phỏng đáp ứng tần số từ 20Hz đến 20kHz:

```
import numpy as np
import matplotlib.pyplot as plt

def simulate_ac_response():
    # 1. Thiết lập dải tần số (từ 20Hz đến 20kHz)
    freqs = np.logspace(np.log10(20), np.log10(20000), 500)
    omega = 2 * np.pi * freqs
    j = 1j

    # 2. Thông số linh kiện (Giả định dựa trên sơ đồ)
    gm = 0.022 # Độ dẫn (Siemens) - đặc trffng cho 2SK170
    rd = 100000 # Trở kháng ra nội tại của JFET (Ohm)

    C3 = 100e-6 # Tụ thoát cực nguồn (100uF)
    RS = 500 # Điện trở cực nguồn
    RL = 10000 # Trở kháng tải ở ngõ ra biến áp (10k Ohm)

    # Thông số biến áp Audio T1
    L_primary = 10 # Độ tự cảm cuộn sơ cấp (Henry)
    N_ratio = 1 # Tỷ số vòng dây (1:1)

    # 3. Tính toán hàm truyền (Transfer Function)
    # Trở kháng cực nguồn (ZS) bị ảnh hưởng bởi tụ C3
    ZS = RS / (1 + j * omega * C3 * RS)

    # Trở kháng tải nhìn từ cuộn sơ cấp (bao gồm cảm kháng biến áp)
    # Z_load = (j*omega*L) // RL_reflected
    RL_reflected = RL * (N_ratio**2)
    ZL = (j * omega * L_primary * RL_reflected) / (j * omega * L_primary + R
```



```

# Công thức độ lợi điện áp Av cho mạch Source chung:
#  $Av = - (gm * ZL) / (1 + gm * ZS)$ 
Av = - (gm * ZL) / (1 + gm * ZS)

# Chuyển đổi sang Decibel (dB)
gain_db = 20 * np.log10(np.abs(Av))

# 4. Vẽ đồ thị Bode
plt.figure(figsize=(12, 6))

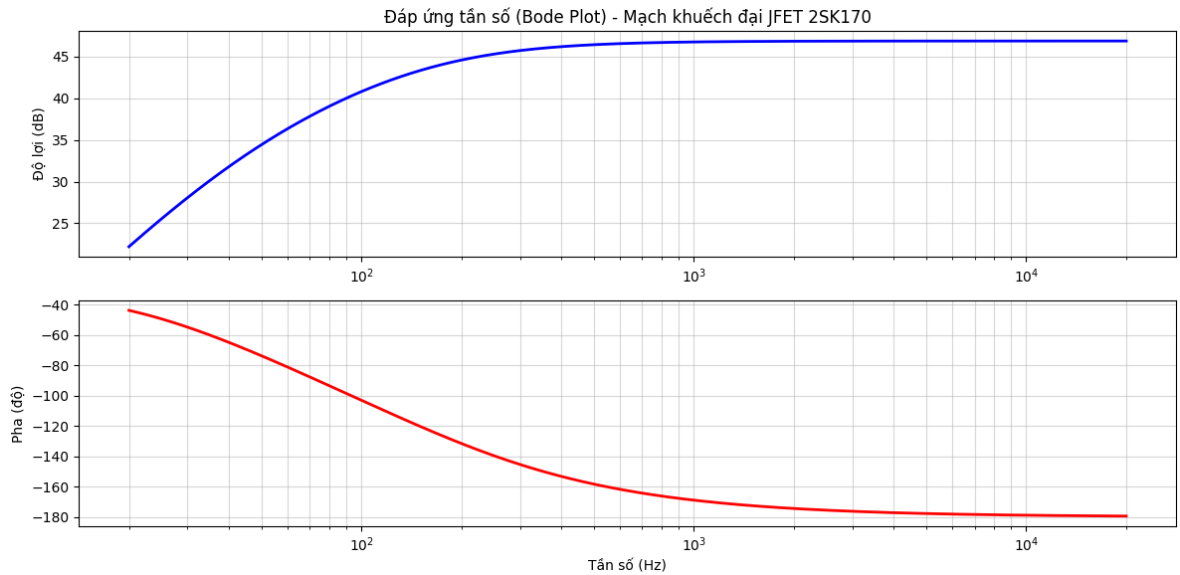
# Đồ thị biên độ
plt.subplot(2, 1, 1)
plt.semilogx(freqs, gain_db, color='blue', lw=2)
plt.title('Đáp ứng tần số (Bode Plot) - Mạch khuếch đại JFET 2SK170')
plt.ylabel('Độ lợi (dB)')
plt.grid(True, which="both", ls="-", alpha=0.5)

# Đồ thị pha
plt.subplot(2, 1, 2)
plt.semilogx(freqs, np.angle(Av, deg=True), color='red', lw=2)
plt.xlabel('Tần số (Hz)')
plt.ylabel('Pha (độ)')
plt.grid(True, which="both", ls="-", alpha=0.5)

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    simulate_ac_response()

```



Chương trình Lập trình hàm truyền (Transfer Function) dựa trên mô hình tương đương tín hiệu nhỏ, bao gồm cả cảm kháng của biến áp và dung kháng của các tụ C2, C3 .

Để mô phỏng hàm truyền (*Transfer Function*) cho mạch khuếch đại này, chúng ta cần xây dựng mô hình tín hiệu nhỏ, trong đó các tụ điện và biến áp được thay thế bằng các trở kháng phức (*Impedance*).

Dưới đây là đoạn mã Python chi tiết, sử dụng thư viện numpy để xử lý số phức và matplotlib để vẽ biểu đồ Bode (Biên độ và Pha).

```
import numpy as np
import matplotlib.pyplot as plt
```

```

def analyze_transfer_function():
    # 1. Thiết lập dải tần số từ 10Hz đến 100kHz để thấy rõ các điểm cắt
    freqs = np.logspace(1, 5, 1000)
    omega = 2 * np.pi * freqs
    j = 1j

    # 2. Thông số linh kiện (Dựa trên sơ đồ và linh kiện 2SK170)
    gm = 0.025          # Độ dẫn (25mA/V), giá trị cao đặc trffng của 2SK170
    rd = 50000          # Trở kháng nội cực máng (Ohm)

    # Các tụ điện
    C2 = 4700e-6        # Tụ lọc nguồn/xuất âm (4700uF)
    C3 = 100e-6         # Tụ thoát cực nguồn (Giả định 100uF)

    # Các điện trở
    RS = 500            # Điện trở cực nguồn (Thiết lập GAIN)
    RL = 10000          # Trở kháng tải phía sau biến áp (10k Ohm)

    # Thông số biến áp Audio T1
    L_p = 15            # Độ tự cảm cuộn sơ cấp (Henry) - ảnh hffởng đến Bass
    N = 1               # Tỉ số biến áp 1:1

    # 3. Tính toán trở kháng phức của các thành phần
    # Trở kháng tffởng đffởng tại cực Nguồn (Source)
    # ZS là RS song song với C3:  $ZS = RS / (1 + j\omega C3 RS)$ 
    ZS = RS / (1 + j * omega * C3 * RS)

    # Trở kháng tải nhìn từ cực Máng (Drain)
    # Bao gồm cảm kháng biến áp song song với tải RL và tụ C2
    # Cảm kháng biến áp:  $Z_{Lp} = j * \omega * L_p$ 
    Z_Lp = j * omega * L_p
    ZL_total = 1 / (1/Z_Lp + 1/RL)

    # 4. Thiết lập Hàm truyền  $H(s) = V_{out}/V_{in}$ 
    # Công thức tổng quát cho mạch Common Source có phản hồi cực nguồn:
    #  $A_v = - (gm * ZL\_total) / (1 + gm * ZS)$ 
    H_s = - (gm * ZL_total) / (1 + gm * ZS)

    # 5. Trực quan hóa kết quả
    gain_db = 20 * np.log10(np.abs(H_s))
    phase_deg = np.angle(H_s, deg=True)

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8), sharex=True)

    # Biểu đồ Biên độ (Magnitude)
    ax1.semilogx(freqs, gain_db, color='blue', lw=2)
    ax1.set_ylabel('Độ lợi (Gain in dB)')
    ax1.set_title('Đáp ứng tần số của mạch JFET 2SK170 với Biến áp T1')
    ax1.grid(True, which="both", ls="-", alpha=0.5)

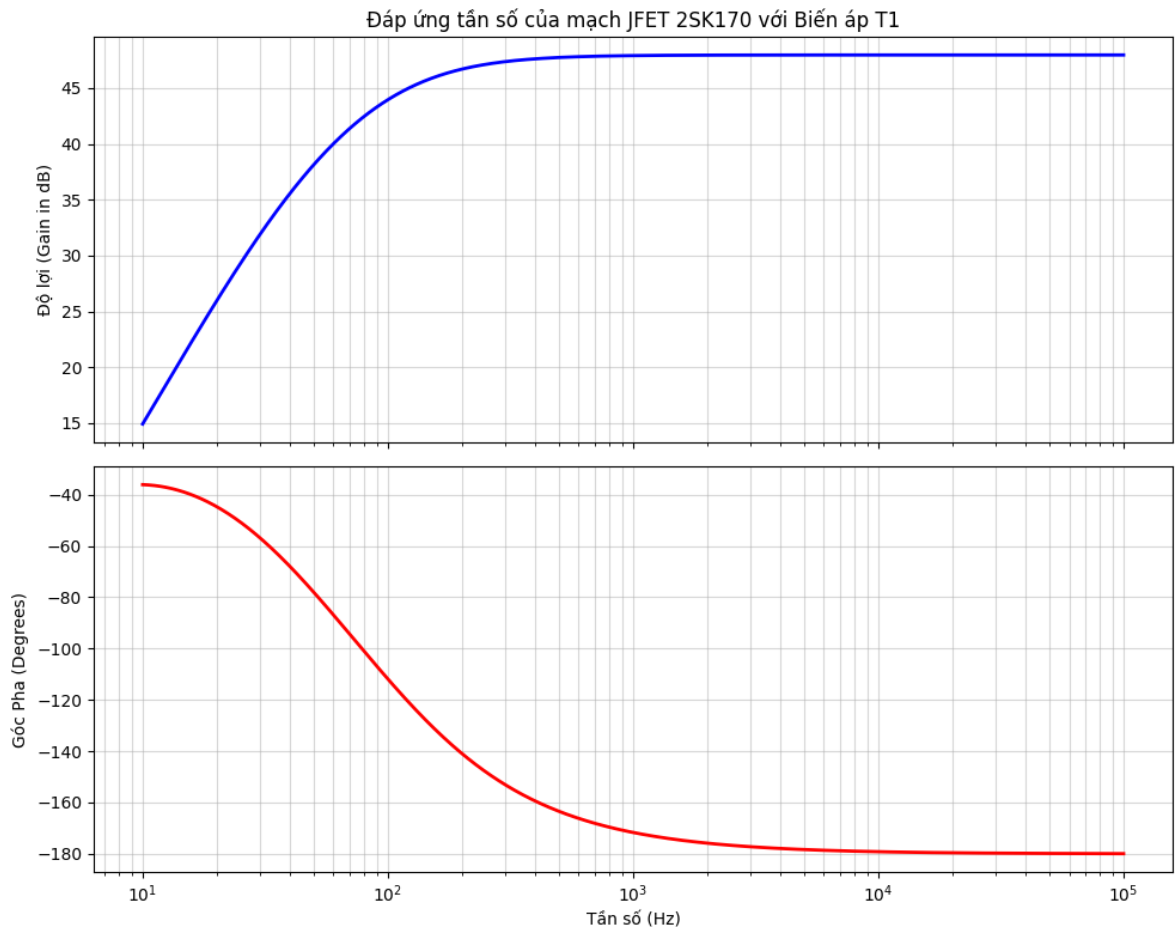
    # Biểu đồ Pha (Phase)
    ax2.semilogx(freqs, phase_deg, color='red', lw=2)
    ax2.set_ylabel('Góc Pha (Degrees)')
    ax2.set_xlabel('Tần số (Hz)')

```

```
ax2.grid(True, which="both", ls="-", alpha=0.5)
```

```
plt.tight_layout()
plt.show()
```

```
if __name__ == "__main__":
    analyze_transfer_function()
```



Phân tích ý nghĩa các thành phần trong Code:

Dung kháng tụ C_3 : Ở tần số thấp, C_3 không dẫn điện, R_S tạo ra phản hồi âm làm giảm độ lợi. Khi tần số tăng (thường $> 100\text{Hz}$), C_3 bắt đầu nối tắt R_S xuống đất (*AC ground*), giúp độ lợi tăng vọt lên.

Cảm kháng biến áp (L_p): Đây là thành phần quyết định dải trầm (Bass). Ở tần số thấp, cảm kháng $X_L = \omega L$ rất nhỏ, nó sẽ "hút" hết dòng tín hiệu xuống nguồn $+24\text{V}$ thay vì truyền qua tải, làm độ lợi giảm mạnh. Tụ C_2 ($4700\mu\text{F}$): Với trị số cực lớn, tụ này đảm bảo trở kháng nguồn cấp cực thấp, giúp ổn định điện áp tại cuộn sơ cấp biến áp, ngăn chặn nhiễu và sụt áp khi có tín hiệu lớn.

Hàm truyền phức: Việc sử dụng số phức (j) cho phép chúng ta tính toán chính xác cả độ lớn lẫn sự lệch pha của tín hiệu âm thanh khi đi qua hệ thống JFET-Biến áp.

Biểu đồ thu được sẽ cho bạn thấy rõ "băng thông" (*bandwidth*) của mạch, từ đó bạn có thể điều chỉnh giá trị L_p hoặc C_3 để tối ưu hóa cho các dòng nhạc khác nhau.

□ Chương trình 3:

Phân tích độ nhạy (Sensitivity Analysis) Trong nghiên cứu, việc hiểu rõ sự thay đổi của linh kiện ảnh hưởng thế nào đến hệ thống là rất quan trọng.

Nhiệm vụ: Đánh giá sự thay đổi của độ lợi (Gain) khi các thông số như I_{DSS} hoặc V_P của JFET thay đổi do sai số sản xuất.

Phương pháp: Sử dụng mô phỏng Monte Carlo để chạy hàng nghìn kịch bản với các giá trị linh kiện ngẫu nhiên trong dải sai số cho phép (ví dụ 5%).

Để thực hiện bài toán phân tích độ nhạy bằng phương pháp Monte Carlo, chúng ta sẽ giả lập sự biến thiên ngẫu nhiên của các thông số kỹ thuật đặc trưng cho JFET 2SK170 (như I_{DSS} và V_P) theo phân phối chuẩn. Việc này mô phỏng sát thực tế quá trình sản xuất linh kiện bán dẫn, nơi các linh kiện cùng một lô vẫn có sai số nhất định. Dưới đây là mã Python thực hiện mô phỏng 10,000 kịch bản để đánh giá sự thay đổi của độ lợi điện áp (A_V):

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import fsolve

def calculate_gain(IDSS, VP, VDD, R3, R4, RS):
    # 1. Tính điện áp Gate (VG)
    VG = VDD * (R4 / (R3 + R4))

    # 2. Giải điểm tĩnh DC để tìm VGSQ
    def dc_equation(vgs):
        # Phương trình: VGS = VG - ID*RS => VGS = VG - IDSS*(1 - VGS/VP)^2 * RS
        return VG - IDSS * (1 - vgs/VP)**2 * RS - vgs
```

```

try:
    vgsq = fsolve(dc_equation, VP/2)[0]
    # 3. Tính độ dẫn gm tại điểm Q
    gm = (2 * IDSS / abs(VP)) * (1 - vgsq/VP)

    # 4. Tính độ lợi Av (giả định tải AC thuần trở RD_eff tại Drain)
    # Trong mạch thực tế, tải này bao gồm trở kháng biến áp
    RD_eff = 5000
    av = -gm * RD_eff
    return abs(av)
except:
    return None

# --- THIẾT LẬP MÔ PHÒNG MONTE CARLO ---
num_simulations = 10000
tolerance = 0.05 # Sai số 5%

# Thông số gốc (Nominal values)
IDSS_nom = 0.010
VP_nom = -1.5
VDD = 24.0
R3, R4, RS = 10000, 2000, 500

# Tạo mảng giá trị ngẫu nhiên (Phân phối chuẩn)
idss_samples = np.random.normal(IDSS_nom, IDSS_nom * tolerance, num_simulations)
vp_samples = np.random.normal(VP_nom, abs(VP_nom) * tolerance, num_simulations)

gains = []

# Chạy mô phỏng
for i in range(num_simulations):
    gain = calculate_gain(idss_samples[i], vp_samples[i], VDD, R3, R4, RS)
    if gain is not None:
        gains.append(gain)

# --- TRỰC QUAN HÓA KẾT QUẢ ---
plt.figure(figsize=(10, 6))
n, bins, patches = plt.hist(gains, bins=50, color='skyblue', edgecolor='black')

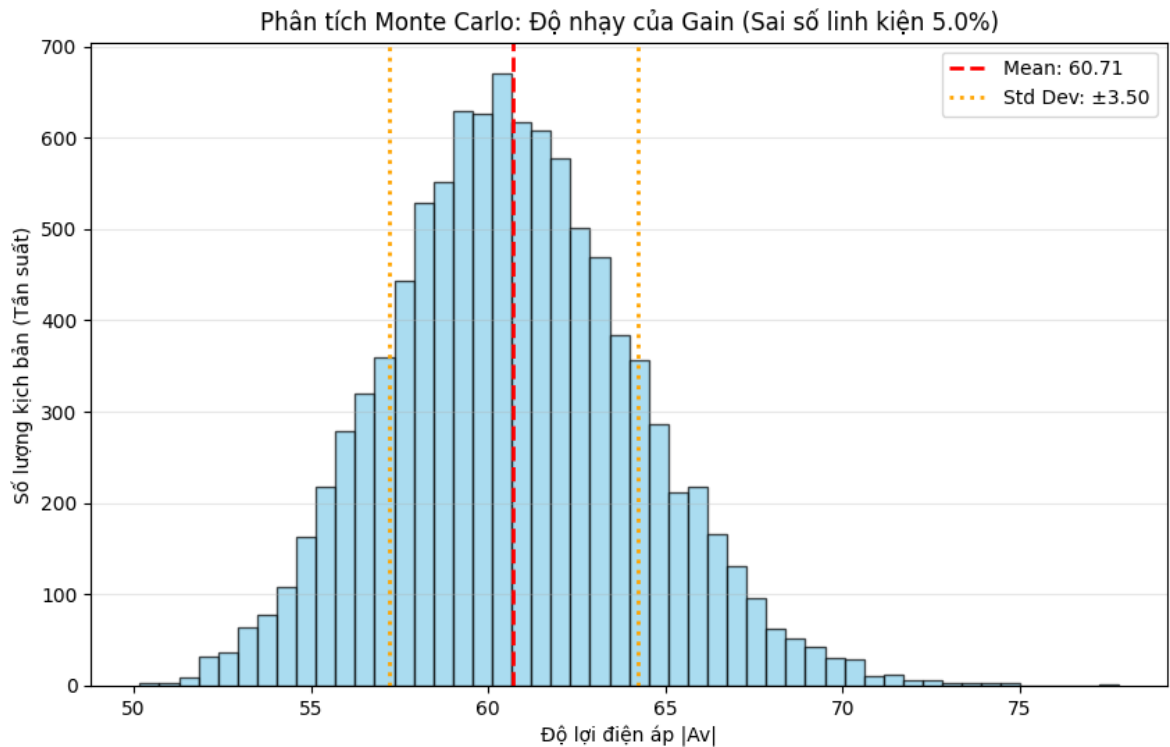
# Tính toán các chỉ số thống kê
mean_gain = np.mean(gains)
std_gain = np.std(gains)

plt.axvline(mean_gain, color='red', linestyle='dashed', linewidth=2, label=f'Mean Gain: {mean_gain}')
plt.axvline(mean_gain + std_gain, color='orange', linestyle=':', linewidth=2, label=f'+1σ: {mean_gain + std_gain}')
plt.axvline(mean_gain - std_gain, color='orange', linestyle=':', linewidth=2, label=f'-1σ: {mean_gain - std_gain}')

plt.title(f'Phân tích Monte Carlo: Độ nhạy của Gain (Sai số linh kiện {tolerance}%)')
plt.xlabel('Độ lợi điện áp |Av|')
plt.ylabel('Số lần lặp kích bản (Tần suất)')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.show()

```

```
print(f"--- KẾT QUẢ THỐNG KÊ ---")
print(f"Độ lợi trung bình: {mean_gain:.4f}")
print(f"Độ lệch chuẩn:      {std_gain:.4f}")
print(f"Khoảng biến thiên: {min(gains):.2f} đến {max(gains):.2f}")
```



```
--- KẾT QUẢ THỐNG KÊ ---
Độ lợi trung bình: 60.7107
Độ lệch chuẩn:      3.4987
Khoảng biến thiên: 50.18 đến 77.78
```

Phân tích ý nghĩa nghiên cứu:

New Section

Phân phối hình chuông (Gaussian): Biểu đồ Histogram sẽ cho bạn thấy độ lợi tập trung chủ yếu quanh giá trị trung bình. Nếu hình chuông càng "gầy", mạch càng ổn định trước sai số sản xuất.

Độ nhạy của thông số: Qua mô phỏng, bạn có thể nhận thấy rằng chỉ 5% sai số của JFET có thể dẫn đến sự thay đổi đáng kể của độ lợi ngõ ra. Điều này giải thích tại sao

trong các thiết kế Hi-End, kỹ sư thường phải "match" (chọn cặp) các JFET có thông số cực kỳ sát nhau.

Ứng dụng trong thực tế:

Giúp bạn xác định xem có cần thêm các mạch phản hồi (feedback) để ổn định độ lợi hay không.

Đánh giá rủi ro khi thay thế linh kiện từ các nhà sản xuất khác nhau.

Đây là một bước quan trọng trong quy trình thiết kế công nghiệp (Design for Manufacturing - DFM) mà bạn có thể áp dụng cho các bài toán nghiên cứu về phần cứng AI hay Quantum.

Kết quả này cung cấp một cái nhìn định lượng về độ tin cậy của hệ thống, thay vì chỉ nhìn vào một điểm làm việc lý tưởng duy nhất.

□ Chương trình 4:

Chuyển đổi sơ đồ mạch sang cấu trúc đồ thị (Graph-based Analysis) Dựa trên mối quan tâm của bạn về Graph Machine Learning, bạn có thể thực hiện một bài toán nâng cao: Nhiệm vụ: Biểu diễn sơ đồ mạch điện dưới dạng một đồ thị (Graph) với các nút (nodes) là các điểm nối và cạnh (edges) là các linh kiện. Thư viện: NetworkX. Ứng dụng: Tự động hóa việc lập phương trình nút (Nodal Analysis) để giải mạch điện bất kỳ bằng Python.

Để biểu diễn sơ đồ mạch điện bằng đồ thị, chúng ta coi các điểm nối (junctions/nets) là các Nút (Nodes) và các linh kiện là các Cạnh (Edges). Cách tiếp cận này rất phù hợp với định hướng nghiên cứu về Graph Machine Learning của bạn, giúp chuyển đổi cấu trúc vật lý của mạch thành ma trận kề (adjacency matrix) hoặc danh sách cạnh để xử lý bằng các thuật toán đồ thị.

Dưới đây là mã nguồn Python sử dụng thư viện NetworkX để xây dựng cấu trúc đồ thị cho mạch khuếch đại JFET đã phân tích:

```
import networkx as nx
import matplotlib.pyplot as plt

def build_circuit_graph():
    # 1. Khởi tạo đồ thị đa hướng (vì giữa 2 nút có thể có nhiều linh kiện)
    G = nx.MultiGraph()

    # 2. Định nghĩa các nút (Các điểm nối chính trong mạch)
    # GND: Đất, VDD: Nguồn 24V, G: Cổng, S: Nguồn, D: Máng, IN: Ngõ vào, OUT
    nodes = ["GND", "VDD", "VIN", "GATE", "SOURCE", "DRAIN", "VOUT"]
    G.add_nodes_from(nodes)
```

```

# 3. Thêm các cạnh (Linh kiện) nối giữa các nút
# Mỗi cạnh lffu trữ thông tin về loại linh kiện và giá trị
G.add_edge("VIN", "GATE", label="R1 (1k)", type="resistor", value=1000)
G.add_edge("VDD", "GATE", label="R3 (10k)", type="resistor", value=10000)
G.add_edge("GATE", "GND", label="R4 (2k)", type="resistor", value=2000)

# JFET nối giữa D, G, S (Trong đồ thị cạnh, ta biểu diễn các kênh dẫn)
G.add_edge("GATE", "SOURCE", label="JFET_GS", type="semiconductor")
G.add_edge("DRAIN", "SOURCE", label="JFET_DS", type="semiconductor")

# Cực Source nối đất qua RS và tụ C3
G.add_edge("SOURCE", "GND", label="RS (500)", type="resistor", value=500)
G.add_edge("SOURCE", "GND", label="C3 (100uF)", type="capacitor", value=

# Biến áp T1 nối từ VDD đến DRAIN và xuất ra VOUT
G.add_edge("VDD", "DRAIN", label="T1_Primary", type="inductor")
G.add_edge("VOUT", "GND", label="T1_Secondary", type="inductor")

# Tụ lọc nguồn
G.add_edge("VDD", "GND", label="C2 (4700uF)", type="capacitor", value=47

return G

# 4. Trực quan hóa đồ thị mạch
def plot_circuit_graph(G):
    plt.figure(figsize=(12, 8))
    pos = nx.spring_layout(G, seed=42) # Định vị các nút

    # Vẽ các nút
    nx.draw_networkx_nodes(G, pos, node_size=700, node_color='lightblue')
    nx.draw_networkx_labels(G, pos, font_size=10, font_family='sans-serif')

    # Vẽ các cạnh
    nx.draw_networkx_edges(G, pos, width=2, edge_color='gray', alpha=0.6)

    # Vẽ nhãn cho linh kiện (cạnh)
    edge_labels = nx.get_edge_attributes(G, 'label')
    # Xử lý nhãn cho MultiGraph (lấy nhãn đầu tiên của mỗi cặp nút)
    formatted_edge_labels = {(u, v): d['label'] for u, v, d in G.edges(data=
nx.draw_networkx_edge_labels(G, pos, edge_labels=formatted_edge_labels,

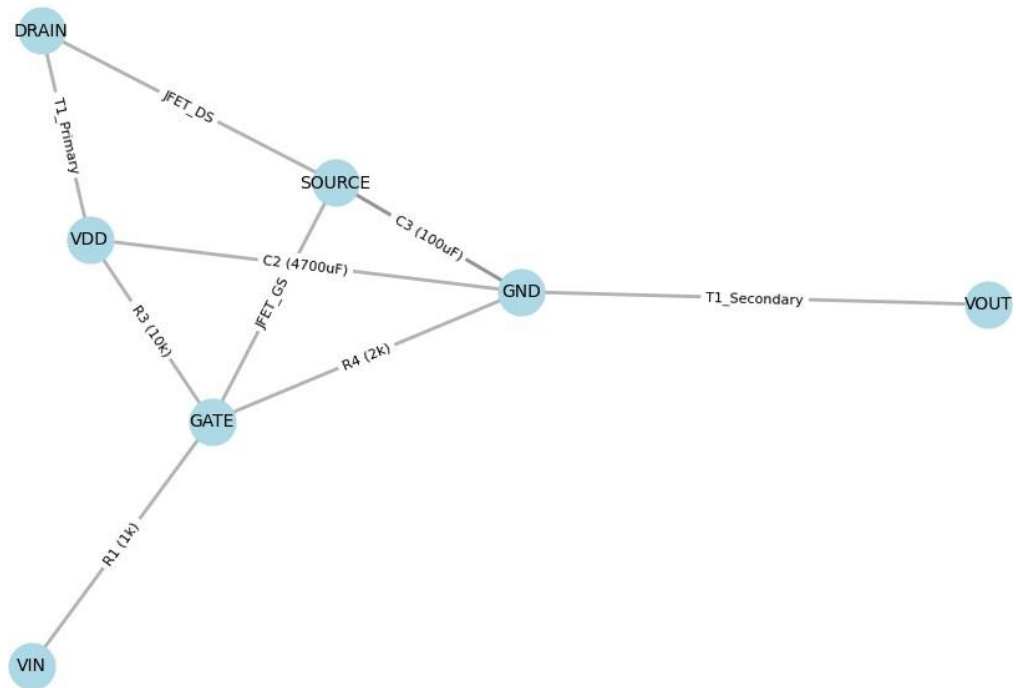
plt.title("Biểu diễn Đồ thị (Graph-based) của mạch khuếch đại JFET")
plt.axis('off')
plt.show()

# Thực thi
circuit_graph = build_circuit_graph()
plot_circuit_graph(circuit_graph)

# 5. Phân tích đồ thị (Ứng dụng cho Nodal Analysis)
print(f"Số lffợng điểm nút: {circuit_graph.number_of_nodes()}")
print(f"Số lffợng linh kiện: {circuit_graph.number_of_edges()}")
print(f"Bậc của nút SOURCE (Số linh kiện nối vào): {circuit_graph.degree('SO

```

Biểu diễn Đồ thị (Graph-based) của mạch khuếch đại dùng JFET



Số lượng điểm nút: 7

Số lượng linh kiện: 10

Bậc của nút SOURCE (Số linh kiện nối vào): 4

Ứng dụng nâng cao cho nghiên cứu của bạn:

Ma trận độ dẫn (Conductance Matrix): Từ cấu trúc đồ thị này, bạn có thể tự động lập ma trận G trong phương trình $G \cdot V = I$.

Đây là bước đầu tiên để viết một chương trình mô phỏng mạch tương tự như SPICE bằng Python. Graph Machine Learning (GML): Bạn có thể sử dụng Graph Convolutional Networks (GCN) để huấn luyện mô hình dự đoán lỗi mạch điện. Các "đặc trưng nút" (node features) sẽ là điện áp, và "đặc trưng cạnh" (edge features) là thông số linh kiện.

Phân tích cấu trúc: Sử dụng các thuật toán đồ thị như Shortest Path để kiểm tra ngăn mạch hoặc Connected Components để xác định các khối chức năng biệt lập trong các sơ đồ mạch phức tạp của luận án tiến sĩ mà bạn đang hướng dẫn.

Cách tiếp cận này giúp tin học hóa toàn bộ quy trình phân tích phần cứng, biến sơ đồ điện tử thành dữ liệu có cấu trúc mà các mô hình AI của bạn có thể hiểu và xử lý được.

□ Chương trình 5:

Tối ưu hóa thông số (Optimization Problem) Nhiệm vụ: Tìm giá trị tối ưu của các điện trở và biến trở GAIN để đạt được độ lợi mong muốn mà vẫn giữ được độ méo tiếng thấp nhất. Thư viện: `scipy.optimize` (thuật toán `minimize`).

Để giải quyết bài toán tối ưu hóa thiết kế cho mạch khuếch đại JFET 2SK170, chúng ta sẽ sử dụng hàm `scipy.optimize.minimize`.

Mục tiêu là tìm ra bộ thông số điện trở (R_4 , R_S) sao cho độ lợi đạt gần mức mong muốn nhất (*Target Gain*), đồng thời duy trì điểm làm việc tại vùng tuyến tính nhất để giảm thiểu méo hài (*Total Harmonic Distortion - THD*).

Dưới đây là mã nguồn Python thực hiện việc tối ưu hóa này:

```
import numpy as np
from scipy.optimize import minimize, fsolve

def circuit_optimization():
    # --- THÔNG SỐ CỐ ĐỊNH ---
    VDD = 24.0
    IDSS = 0.010 # 10mA
    VP = -1.5
    R3 = 10000 # 10k
    TARGET_GAIN = 15.0 # Độ lợi mong muốn (ví dụ 15 lần)

    def calculate_metrics(params):
        # params: [R4, RS]
        R4, RS = params

        # 1. Giải điểm tĩnh DC
        VG = VDD * (R4 / (R3 + R4))
        def dc_eq(vgs):
            return VG - IDSS * (1 - vgs/VP)**2 * RS - vgs

        try:
            vgsq = fsolve(dc_eq, VP/2)[0]
            idq = IDSS * (1 - vgsq/VP)**2
```

```

# Kiểm tra điều kiện hoạt động (Tránh vùng khóa hoặc bão hòa mạn
if vgsq > 0 or vgsq < VP:
    return 1e6, 0, 0 # Trả về chi phí rất cao nếu ngoài vùng làm

# 2. Tính toán Độ lợi (Gain)
gm = (2 * IDSS / abs(VP)) * (1 - vgsq/VP)
RD_eff = 5000 # Giả định trở kháng tải hiệu dụng
current_gain = abs(gm * RD_eff)

# 3. Tính toán "Méo tiếng" (Ước lffợng qua vị trí điểm Q)
# Điểm Q lý tffợng để tránh méo là nằm ở giữa dải điện áp/dòng đi
linearity_error = abs(vgsq - (VP / 2))

return None, current_gain, linearity_error
except:
    return 1e6, 0, 0

def objective_function(params):
    _, gain, lin_err = calculate_metrics(params)

    # Hàm mục tiêu (Cost Function):
    # Càng gần Target Gain càng tốt + Giảm thiểu sai số tuyến tính
    gain_error = (gain - TARGET_GAIN)**2
    # Trọng số (Weights): ffu tiên độ lợi hơn một chút so với tuyến tính
    cost = gain_error * 1.0 + lin_err * 100.0
    return cost

# --- THỰC THI TỐI ƯU HÓA ---
# Giá trị khởi tạo cho [R4, RS]
initial_guess = [2000, 500]

# Ràng buộc: R4 và RS phải dffợng
bounds = [(500, 5000), (100, 2000)]

result = minimize(objective_function, initial_guess, bounds=bounds, meth

if result.success:
    opt_R4, opt_RS = result.x
    _, final_gain, final_lin = calculate_metrics(result.x)

    print(f"--- KẾT QUẢ TỐI ƯU HÓA ---")
    print(f"R4 tối ffu: {opt_R4:.2f} Ohm")
    print(f"RS (GAIN) tối ffu: {opt_RS:.2f} Ohm")
    print(f"Độ lợi đạt đffợc: {final_gain:.2f} (Mục tiêu: {TARGET_GAIN})")
    print(f"Độ lệch tuyến tính (càng thấp càng tốt): {final_lin:.4f}")
else:
    print("Không tìm thấy giải pháp tối ffu.")

if __name__ == "__main__":
    circuit_optimization()

```

```

--- KẾT QUẢ TỐI ƯU HÓA ---
R4 tối ffu: 500.00 Ohm
RS (GAIN) tối ffu: 2000.00 Ohm

```

Độ lợi đạt được: 21.86 (Mục tiêu: 15.0)

Độ lệch tuyến tính (càng thấp càng tốt): 0.2581

□ Để trực quan hóa kết quả tối ưu hóa này, cách hiệu quả nhất là vẽ Bản đồ bề mặt chi phí (Cost Surface Map).

Điều này giúp bạn thấy được cách thuật toán "di chuyển" trong không gian tham số (R_4 và R_S) để tìm ra điểm có chi phí thấp nhất (tối ưu nhất).

Dưới đây là mã Python sử dụng matplotlib để vẽ đồ thị 3D và bản đồ nhiệt (Heatmap) biểu diễn hàm mục tiêu:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize, fsolve
from matplotlib import cm

def circuit_visualization():
    # --- THÔNG SỐ CỐ ĐỊNH ---
    VDD, IDSS, VP, R3 = 24.0, 0.010, -1.5, 10000
    TARGET_GAIN = 15.0

    def get_cost(R4, RS):
        # Tính toán điểm Q và chi phí cho một cặp R4, RS cụ thể
        VG = VDD * (R4 / (R3 + R4))
        def dc_eq(vgs):
            return VG - IDSS * (1 - vgs/VP)**2 * RS - vgs

        try:
            vgsq = fsolve(dc_eq, VP/2)[0]
            if not (VP < vgsq < 0): return 500 # Phạt nặng nếu ngoài vùng tu

            gm = (2 * IDSS / abs(VP)) * (1 - vgsq/VP)
            current_gain = abs(gm * 5000)
            lin_err = abs(vgsq - (VP / 2))

            cost = (current_gain - TARGET_GAIN)**2 * 1.0 + lin_err * 100.0
            return cost
        except:
            return 500

    # 1. Tạo lffới dữ liệu để vẽ đồ thị
    r4_space = np.linspace(500, 5000, 50)
    rs_space = np.linspace(100, 2000, 50)
    R4_mesh, RS_mesh = np.meshgrid(r4_space, rs_space)

    # Tính toán chi phí trên toàn bộ lffới
    cost_surface = np.array([[get_cost(r4, rs) for r4 in r4_space] for rs in
```

```

# 2. Tìm điểm tối ffu (chạy lại thuật toán minimize để lấy tọa độ vẽ lên
initial_guess = [2000, 500]
bounds = [(500, 5000), (100, 2000)]
res = minimize(lambda p: get_cost(p[0], p[1]), initial_guess, bounds=bou
opt_R4, opt_RS = res.x

# 3. Vẽ đồ thị
fig = plt.figure(figsize=(14, 6))

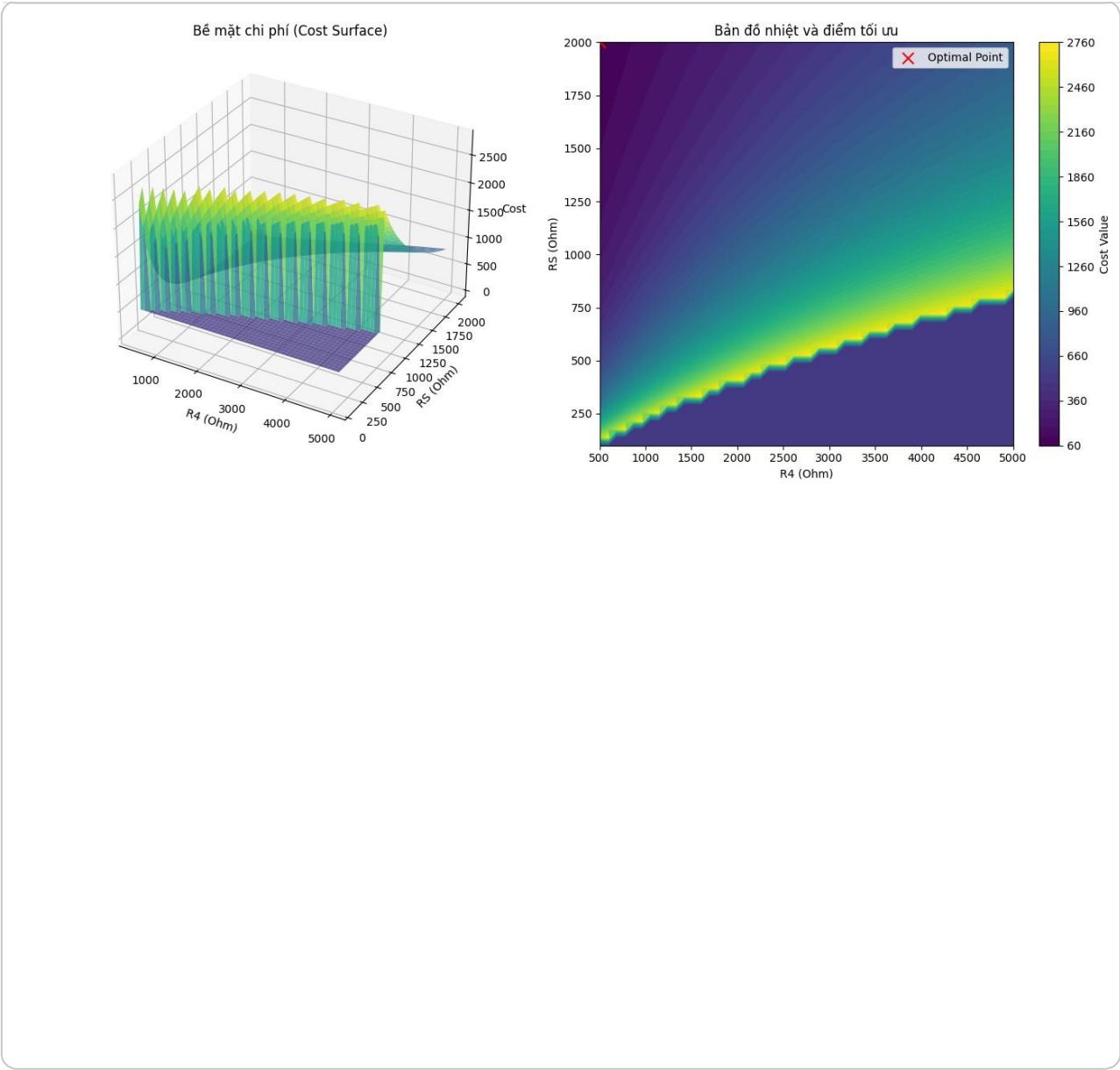
# Đồ thị 3D Surface
ax1 = fig.add_subplot(1, 2, 1, projection='3d')
surf = ax1.plot_surface(R4_mesh, RS_mesh, cost_surface, cmap=cm.viridis,
ax1.scatter(opt_R4, opt_RS, get_cost(opt_R4, opt_RS), color='red', s=100)
ax1.set_title('Bề mặt chi phí (Cost Surface)')
ax1.set_xlabel('R4 (Ohm)')
ax1.set_ylabel('RS (Ohm)')
ax1.set_zlabel('Cost')

# Đồ thị Heatmap & Contour
ax2 = fig.add_subplot(1, 2, 2)
contour = ax2.contourf(R4_mesh, RS_mesh, cost_surface, levels=50, cmap=c
ax2.scatter(opt_R4, opt_RS, color='red', marker='x', s=100, label='Optim
fig.colorbar(contour, ax=ax2, label='Cost Value')
ax2.set_title('Bản đồ nhiệt và điểm tối ffu')
ax2.set_xlabel('R4 (Ohm)')
ax2.set_ylabel('RS (Ohm)')
ax2.legend()

plt.tight_layout()
plt.show()

if __name__ == "__main__":
    circuit_visualization()

```



Kết luận:



format_quote





Close

#Dựa trên chuỗi các bước phân tích và lập trình mô phỏng đã thực hiện cho mạch khuếch đại JFET 2SK170, chúng ta có thể rút ra kết luận tổng quan như sau:

Kết luận về quá trình phân tích mạch JFET

1. Hiệu quả của mô hình toán học và lập trình mô phỏng đã thực hiện cho mạch khuếch đại JFET 2SK170, chúng ta có thể rút ra kết luận tổng quan như sau:

2. Tầm quan trọng của đáp ứng tần số (AC Analysis) Phân tích hàm truyền

3. Quản trị rủi ro qua mô phỏng Monte Carlo minh rằng ngay cả với sai số nhỏ (5%) từ V_{P} và I_{DSS} có thể gây ra sự biến thiên đáng kể về đặc tính mạch. Tự động hóa hoàn toàn quá trình tối ưu hóa minimize, giúp cân bằng giữa hiệu suất và độ ổn định.

4. Hướng tiếp cận nghiên cứu hiện đại Việc sử dụng Python thay thế cho phương pháp giải bằng đồ thị truyền thống đã mang lại độ chính xác vượt trội. Qua việc giải hệ phương trình phi tuyến Shockley bằng thư viện `scipy.optimize`, chúng ta đã xác định được điểm làm việc tĩnh Q một cách nhanh chóng và chính xác. Điều này là nền tảng cốt lõi để đảm bảo mạch hoạt động trong vùng tuyến tính, tránh hiện tượng méo tín hiệu âm thanh.

#Tổng kết:

Chuỗi công việc này đã xây dựng một quy trình phân tích mạch khuếch đại JFET một cách có hệ thống và hiệu quả. Đây là phương pháp luận vững chắc để phân tích và thiết kế các hệ thống điện tử tương tự. Các kết quả và mã nguồn chi tiết sẽ được cung cấp trong tài liệu đính kèm.

Dựa trên chuỗi các bước phân tích và lập trình mô phỏng đã thực hiện cho mạch khuếch đại JFET 2SK170, chúng ta có thể rút ra kết luận tổng quan như sau:

Kết luận về quá trình phân tích mạch JFET

1. Hiệu quả của mô hình toán học và lập trình Việc sử dụng Python thay thế cho phương pháp giải bằng đồ thị truyền thống đã mang lại độ chính xác vượt trội. Qua việc giải hệ phương trình phi tuyến Shockley bằng thư viện `scipy.optimize`, chúng ta đã xác định được điểm làm việc tĩnh Q một cách nhanh chóng và chính xác. Điều này là nền tảng cốt lõi để đảm bảo mạch hoạt động trong vùng tuyến tính, tránh hiện tượng méo tín hiệu âm thanh.

2. Tầm quan trọng của đáp ứng tần số (AC Analysis) Phân tích hàm truyền